
PROJET DE FIN DE SEMESTRE

Master 1 Informatique — Module : Compilation

Système de Gestion des Notes

**Analyse Lexicale et Syntaxique avec
FLEX, BISON & Interface Python**

Niveau : Master 1 Informatique

Module : Théorie des Langages & Compilation

Outils : Flex 2.6+, Bison 3.x, Python 3.10+, Tkinter/PyQt

Durée : 6 semaines

Travail : Binôme (2 étudiants)

Année : 2025–2026

[!] Important

La date limite de rendu est fixée pour le 15 Juin 2026

Table des matières

1	Présentation Générale du Projet	2
1.1	Contexte et Motivation	2
1.2	Description du Domaine	2
2	Spécification du Langage à Analyser	2
2.1	Structure du Fichier d'Entrée	2
2.2	Grammaire Formelle (BNF)	3
2.3	Tokens Lexicaux	5
3	Travail Demandé	5
3.1	Partie 1 — Analyseur Lexical avec Flex	5
3.1.1	Description	5
3.1.2	Livrables attendus	5
3.2	Partie 2 — Analyseur Syntaxique et Sémantique avec Bison	5
3.2.1	Description	5
3.2.2	Règle de calcul	6
3.2.3	Décision de passage	6
3.3	Partie 3 — Interface Graphique Python	6
3.3.1	Description	6
3.3.2	Format de sortie du binaire	7
3.4	Partie 4 — Rapport Technique	7
4	Architecture du Projet	8
4.1	Organisation des Fichiers	8
4.2	Flux d'Exécution	9
5	Contraintes Techniques	9
6	Barème d'Évaluation	10
7	Conseils et Ressources	10
7.1	Références Bibliographiques	10
7.2	Conseils Pratiques	11
8	Modalités de Rendu	11

1. Présentation Générale du Projet

1.1. Contexte et Motivation

Dans le cadre du module de **Compilation** du Master 1 Informatique, il vous est demandé de concevoir et d'implémenter un **Système de Gestion des Notes (SGN)** pour les trois premières années d'une formation universitaire (Licence 1, Licence 2, Licence 3).

Ce projet a pour but de mettre en pratique les concepts fondamentaux des langages formels et de la compilation :

- La conception d'une **grammaire formelle** pour un langage de saisie structurée,
- L'implémentation d'un **analyseur lexical** avec **Flex**,
- L'implémentation d'un **analyseur syntaxique et sémantique** avec **Bison**,
- La génération d'une **représentation intermédiaire** (AST ou table de symboles),
- La réalisation d'une **interface graphique de test** en **Python**.

[*] Objectif

À la fin de ce projet, l'étudiant sera capable de définir un langage dédié (DSL), d'écrire sa grammaire BNF/EBNF, d'analyser et d'interpréter des fichiers de données structurées, et d'interconnecter un compilateur C avec un frontend Python.

1.2. Description du Domaine

Le système doit gérer les notes d'étudiants réparties sur **trois niveaux d'études** :

Niveau	Désignation	Modules types	Semestres
L1	Première année de Licence	Maths, Algo, Réseaux, Physique	S1, S2
L2	Deuxième année de Licence	POO, BDD, Systèmes, Stats	S3, S4
L3	Troisième année de Licence	Compilation, IA, Génie Logiciel	S5, S6

Chaque étudiant possède : un **matricule unique**, un **nom complet**, un **niveau d'inscription**, et une liste de notes par module avec un **coefficient** associé.

2. Spécification du Langage à Analyser

2.1. Structure du Fichier d'Entrée

Le projet repose sur la définition d'un **langage de description de notes** (fichier `.sgn`). Voici la structure attendue :

Listing 1 – Exemple de fichier `.sgn`

```
1 -- Fichier notes : Annee academique 2025-2026
```

```

2 ANNEE "2025-2026"
3
4 NIVEAU L1 {
5     ETUDIANT {
6         MATRICULE : "L1-2025-001"
7         NOM       : "DIALLO Mamadou"
8         PRENOM    : "Mamadou"
9         SEMESTRE S1 {
10            MODULE "Algorithmique"      COEF 3 NOTE 14.5
11            MODULE "Mathematiques"      COEF 4 NOTE 12.0
12            MODULE "Architecture"       COEF 2 NOTE 16.0
13            MODULE "Anglais"            COEF 1 NOTE 15.0
14        }
15        SEMESTRE S2 {
16            MODULE "Programmation C"    COEF 3 NOTE 13.5
17            MODULE "Logique"            COEF 3 NOTE 11.0
18            MODULE "Reseaux"            COEF 2 NOTE 17.0
19        }
20    }
21    ETUDIANT {
22        MATRICULE : "L1-2025-002"
23        NOM       : "NDIAYE Fatou"
24        PRENOM    : "Fatou"
25        SEMESTRE S1 {
26            MODULE "Algorithmique"      COEF 3 NOTE 9.5
27            MODULE "Mathematiques"      COEF 4 NOTE 8.0
28        }
29    }
30 }
31
32 NIVEAU L2 {
33     ETUDIANT {
34         MATRICULE : "L2-2024-010"
35         NOM       : "SALL Ibrahim"
36         PRENOM    : "Ibrahim"
37         SEMESTRE S3 {
38            MODULE "Bases de Donnees"   COEF 3 NOTE 15.0
39            MODULE "POO Java"           COEF 4 NOTE 13.0
40            MODULE "Systemes"           COEF 3 NOTE 14.0
41        }
42        SEMESTRE S4 {
43            MODULE "Compilation"         COEF 3 NOTE 16.5
44            MODULE "Reseaux Avances"     COEF 3 NOTE 12.5
45        }
46    }
47 }

```

2.2. Grammaire Formelle (BNF)

La grammaire formelle du langage `.sgn` est définie comme suit :

```
<programme>      ::= ANNEE STRING <liste_niveaux>

<liste_niveaux> ::= <niveau> | <liste_niveaux> <niveau>

<niveau>          ::= NIVEAU <id_niveau> '{' <liste_etudiants> '}'

<id_niveau>       ::= L1 | L2 | L3

<liste_etudiants> ::= <etudiant> | <liste_etudiants> <etudiant>

<etudiant>        ::= ETUDIANT '{' <champs_etudiant> <liste_semestres> '}'

<champs_etudiant> ::= MATRICULE ':' STRING
                     NOM ':' STRING
                     PRENOM ':' STRING

<liste_semestres> ::= <semestre> | <liste_semestres> <semestre>

<semestre>        ::= SEMESTRE <id_sem> '{' <liste_modules> '}'

<id_sem>          ::= S1 | S2 | S3 | S4 | S5 | S6

<liste_modules> ::= <module> | <liste_modules> <module>

<module>          ::= MODULE STRING COEF ENTIER NOTE REEL

<STRING>          ::= '"' [^"]* '"'
<REEL>            ::= [0-9]+ ('.' [0-9]+)?
<ENTIER>          ::= [0-9]+
```

2.3. Tokens Lexicaux

Token	Description	Exemple
ANNEE	Mot-clé de début de fichier	ANNEE
NIVEAU	Déclaration d'un niveau	NIVEAU
ETUDIANT	Bloc étudiant	ETUDIANT
MATRICULE	Identifiant matricule	MATRICULE
NOM / PRENOM	Champs nominaux	NOM, PRENOM
SEMESTRE	Déclaration d'un semestre	SEMESTRE
MODULE	Déclaration d'un module	MODULE
COEF	Coefficient du module	COEF
NOTE	Note obtenue	NOTE
L1/L2/L3	Identifiants de niveau	L1, L2, L3
S1..S6	Identifiants de semestre	S1, S4
REEL	Nombre réel	14.5, 9.0
ENTIER	Nombre entier	3, 1
STRING	Chaîne entre guillemets	"Algorithmique"
COMMENTAIRE	Ligne débutant par -	- Commentaire

3. Travail Demandé

3.1. Partie 1 — Analyseur Lexical avec Flex

3.1.1. Description

Vous devez écrire un fichier `lexer.l` reconnaissant l'ensemble des tokens du langage `.sgn`. L'analyseur devra :

- Ignorer les espaces, tabulations, retours à la ligne,
- Ignorer les commentaires (lignes commençant par -),
- Reconnaître les mots-clés réservés (`ANNEE`, `NIVEAU`, `ETUDIANT`, etc.),
- Reconnaître les littéraux de type chaîne, entier et réel,
- Gérer les erreurs lexicales avec message et numéro de ligne.

3.1.2. Livrables attendus

- Fichier `lexer.l` commenté et fonctionnel,
- Tableau de tous les tokens reconnus (dans le rapport),
- Tests de robustesse sur des entrées incorrectes.

3.2. Partie 2 — Analyseur Syntaxique et Sémantique avec Bison

3.2.1. Description

Le fichier `parser.y` devra implémenter :

1. **L'analyse syntaxique** conforme à la grammaire BNF définie à la section 2.2,
2. **La construction d'une structure de données** représentant les informations parsées (table de symboles ou AST),

3. Les vérifications sémantiques suivantes :

- Les notes sont dans l'intervalle $[0.0, 20.0]$,
- Les coefficients sont des entiers strictement positifs (≥ 1),
- Absence de doublon de matricule dans un même niveau,
- Cohérence des semestres par rapport au niveau ($L1 \rightarrow S1, S2$; $L2 \rightarrow S3, S4$; $L3 \rightarrow S5, S6$).

4. Le calcul automatique des indicateurs :

- Moyenne pondérée par semestre,
- Moyenne annuelle,
- Rang dans le niveau,
- Décision de passage (mention).

3.2.2. Règle de calcul

La moyenne pondérée d'un semestre est :

$$\bar{M}_s = \frac{\sum_{i=1}^n c_i \times n_i}{\sum_{i=1}^n c_i} \quad (1)$$

La moyenne annuelle est la moyenne arithmétique des moyennes semestrielles :

$$\bar{M}_a = \frac{1}{S} \sum_{s=1}^S \bar{M}_s \quad (2)$$

3.2.3. Décision de passage

Moyenne Annuelle	Décision	Mention
$M_a \geq 16.0$	Admis	Très Bien
$14.0 \leq M_a < 16.0$	Admis	Bien
$12.0 \leq M_a < 14.0$	Admis	Assez Bien
$10.0 \leq M_a < 12.0$	Admis	Passable
$M_a < 10.0$	Ajourné	—

3.3. Partie 3 — Interface Graphique Python

3.3.1. Description

Une interface graphique doit être développée en Python (avec **Tkinter** ou **PyQt5/PyQt6**). Elle doit permettre :

1. **Chargement de fichier** : bouton pour sélectionner un fichier `.sgn`,
2. **Éditeur de texte intégré** : zone permettant d'éditer le fichier directement,
3. **Lancement de l'analyse** : appel au programme Flex/Bison compilé via `subprocess`,
4. **Affichage des résultats** : tableau synthétique avec :
 - Matricule, Nom, Niveau, Semestre(s),

- Moyenne semestrielle, Moyenne annuelle,
 - Rang dans le niveau, Mention, Décision.
5. **Affichage des erreurs** : zone colorée affichant les erreurs lexicales/syntaxiques/-sémantiques avec le numéro de ligne,
 6. **Export des résultats** : vers un fichier `.csv` ou `.txt`.

[i] Remarque

La communication entre l'interface Python et le binaire Flex/Bison se fait via la **sortie standard** (`stdout/stderr`). Le binaire doit afficher les résultats dans un format structuré (ex. CSV ou JSON) lisible par Python.

3.3.2. Format de sortie du binaire

Le programme Flex/Bison devra écrire sur `stdout` un flux JSON de la forme :

Listing 2 – Format de sortie JSON attendu

```
1 {
2   "annee": "2025-2026",
3   "niveaux": [
4     {
5       "niveau": "L1",
6       "etudiants": [
7         {
8           "matricule": "L1-2025-001",
9           "nom": "DIALLO Mamadou",
10          "semestres": [
11            {
12              "id": "S1",
13              "modules": [
14                {"nom": "Algorithmique", "coef": 3, "note": 14.5}
15              ],
16              "moyenne": 14.07
17            }
18          ],
19          "moyenne_annuelle": 14.07,
20          "mention": "Assez Bien",
21          "decision": "Admis",
22          "rang": 1
23        }
24      ]
25    }
26  ],
27  "erreurs": []
28 }
```

3.4. Partie 4 — Rapport Technique

Le rapport (15 à 25 pages, format PDF) devra contenir :

1. **Introduction** : contexte, objectifs, organisation du travail,
2. **Spécification du langage** : grammaire EBNF complète, diagrammes de transition,
3. **Implémentation Flex** : description des règles lexicales, gestion des erreurs,
4. **Implémentation Bison** : description des règles de production, actions sémantiques,
5. **Structure de données** : description de l'AST ou de la table de symboles,
6. **Interface Python** : captures d'écran, architecture MVC,
7. **Tests et validation** : jeux de tests (nominaux et d'erreur),
8. **Conclusion** : bilan, difficultés rencontrées, perspectives.

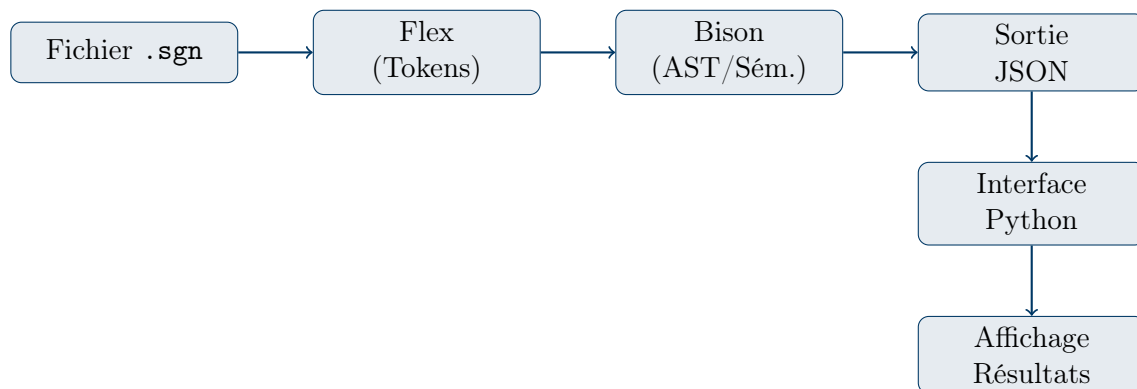
4. Architecture du Projet

4.1. Organisation des Fichiers

Listing 3 – Arborescence du projet

```
1  sgn_project/  
2  |-- src/  
3  |   |-- lexer.l           # Analyseur lexical Flex  
4  |   |-- parser.y          # Analyseur syntaxique Bison  
5  |   |-- ast.h / ast.c     # Structure AST (optionnel)  
6  |   |-- symboles.h        # Table de symboles  
7  |   |-- main.c            # Point d'entree  
8  |   |-- Makefile          # Compilation automatique  
9  |  
10 |-- gui/  
11 |   |-- app.py             # Application Python principale  
12 |   |-- interface.py       # Widgets Tkinter / PyQt  
13 |   |-- parser_bridge.py    # Pont Python <-> Binaire C  
14 |  
15 |-- tests/  
16 |   |-- test_valide.sgn     # Fichier sans erreurs  
17 |   |-- test_erreur_lex.sgn # Erreurs lexicales  
18 |   |-- test_erreur_syn.sgn # Erreurs syntaxiques  
19 |   |-- test_erreur_sem.sgn # Erreurs semantiques  
20 |  
21 |-- rapport/  
22 |   |-- rapport.pdf  
23 |   |-- rapport.tex  
24 |  
25 |-- README.md
```

4.2. Flux d'Exécution



5. Contraintes Techniques

[!] Important

Toutes les contraintes ci-dessous sont **obligatoires**. Le non-respect entraîne une pénalité sur la note finale.

1. Le projet doit **compiler sans erreur** avec `make` sous Linux (Ubuntu/Debian),
2. L'analyseur Flex doit **signaler les erreurs lexicales avec le numéro de ligne**,
3. L'analyseur Bison doit **effectuer la récupération d'erreurs** (`yyerror`),
4. Les **vérifications sémantiques** sont obligatoires (cf. section 3.2.1),
5. L'interface Python doit être **exécutable sans modification** avec `python3 app.py`,
6. Les communications Python ↔ C sont réalisées **uniquement via subprocess et stdout**,
7. Le code doit être **commenté en français** avec en-tête auteur/date,
8. L'**arborescence** du projet doit respecter la structure définie à la section 4.1.

6. Barème d'Évaluation

Critère d'Évaluation	Points	Pondération
Partie 1 : Analyseur Lexical Flex		5 pts
Reconnaissance correcte de tous les tokens	2	
Gestion des erreurs lexicales + numéro de ligne	1.5	
Ignorer commentaires & espaces	1	
Qualité et lisibilité du code	0.5	
Partie 2 : Analyseur Syntaxique Bison		7 pts
Grammaire conforme à la BNF	2	
Construction de la table/structure de données	2	
Vérifications sémantiques complètes	2	
Calculs (moyennes, rang, mention)	1	
Partie 3 : Interface Graphique Python		5 pts
Chargement & édition de fichier	1	
Affichage des résultats en tableau	1.5	
Affichage des erreurs (coloré, numéro ligne)	1	
Export CSV/TXT	1	
Qualité UX & ergonomie	0.5	
Rapport Technique		3 pts
Complétude & structure	1.5	
Qualité de rédaction	1	
Jeux de tests documentés	0.5	
TOTAL	20	100%

7. Conseils et Ressources

7.1. Références Bibliographiques

- **Flex & Bison**, John Levine, O'Reilly, 2009.
- **Compilers : Principles, Techniques, and Tools** (Aho, Lam, Sethi, Ullman), Pearson, 2006.
- Documentation officielle Flex : <https://github.com/westes/flex>
- Documentation officielle Bison : <https://www.gnu.org/software/bison/manual/>
- Python subprocess : <https://docs.python.org/3/library/subprocess.html>

7.2. Conseils Pratiques

[i] Remarque

- Commencez par tester votre analyseur lexical **seul** avant de l'intégrer avec Bison.
- Utilisez `YYDEBUG=1` pour déboguer le parseur Bison.
- Gérez les fuites mémoire avec `valgrind` sur le binaire C.
- En Python, utilisez `subprocess.run()` avec `capture_output=True` pour capturer la sortie JSON.
- Adoptez une approche itérative : fonctionnalités de base d'abord, puis améliorations.

8. Modalités de Rendu

1. L'archive du projet doit être nommée : `NOM1_NOM2_SGN_M1.zip`
2. Elle doit contenir l'intégralité de l'arborescence définie en section 4.1,
3. Un fichier `README.md` doit expliquer comment **compiler et exécuter** le projet,
4. Le rendu se fait sur la **plateforme pédagogique** du département,
5. Une **soutenance de 15 minutes** (10 min démo + 5 min questions) est prévue.

[!] Important

Tout plagiat ou copie sera sanctionné par la note de **0/20** pour les deux binômes concernés.